

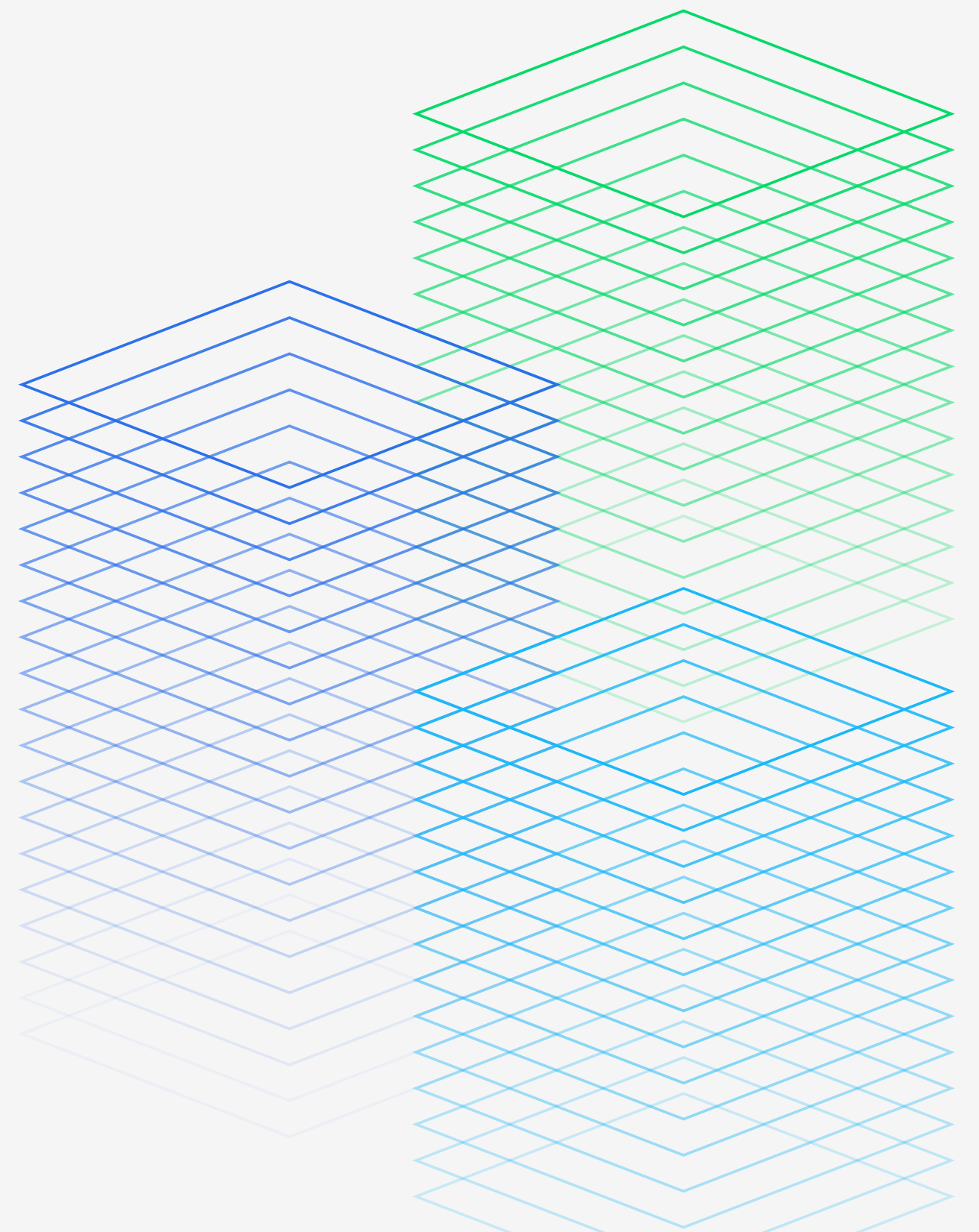
AI 활용 project

최종 발표

Pygame을 활용한 MBTI 검사 및
유형별 정보 탐색 시스템 구현

subject 전자공학기초실험

member 20251511 김다연 | 20251536 이금성나 | 20251543 이지현



주제 선정 배경

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현

부담감 · 진입장벽 ↓ project

생활 속 필요한 맞춤형
조언 제공

일상 속 친숙하게 접할 수 있는
key word 활용

사용자 참여형 시스템 구현 경험

인터페이스 구성, 프로그램 흐름
설계 집중

프로그램 구성

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현

01

유형 검사

시스템 내 해당
점수 체크

02

결과 도출

본인 유형 파악

03

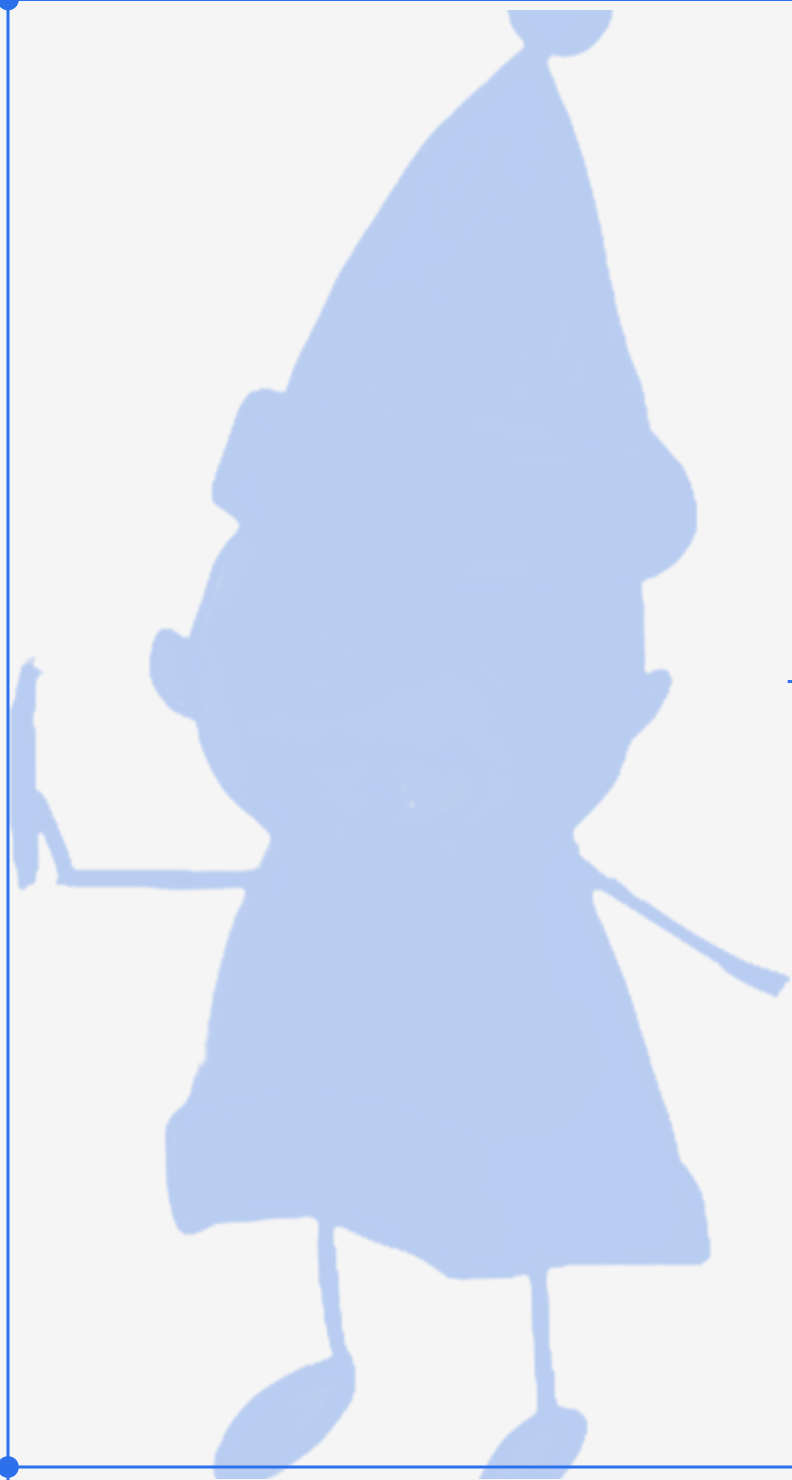
결과 도출

유형별 궁합 확인

04

마무리

유형별 맞춤 조언



수정 전 system

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현



프로젝트 보완 사항

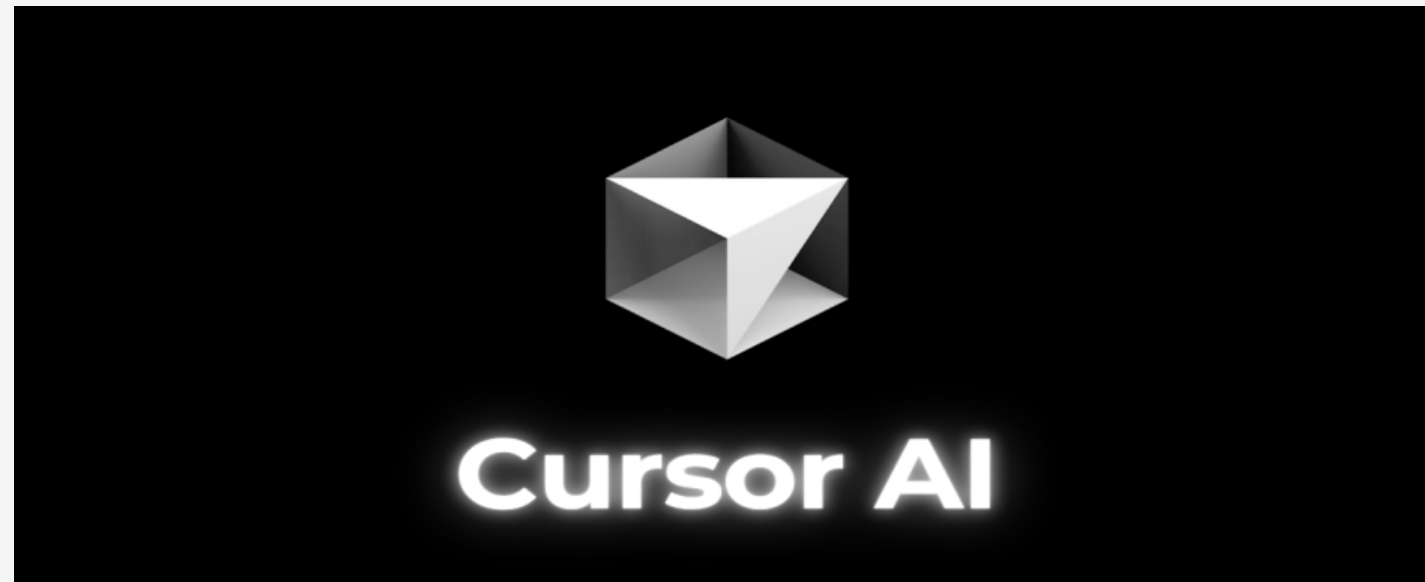
Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현

검사 결과의
직관적 이미지화

응답 데이터
초기화 및 재시작

구상했던 컨셉에 맞는
시각적 UI 구현

사용 AI Tool



Cursor

코드 설계



Gemini

Cursor 설계 코드 축소 및 최적화

기존 기본 코드 설계

01 MBTI 질문 디셔너리

→ 지표 별 질문 리스트 형태 저장

02 점수 계산 function

→ raw_max, raw_min
퍼센트 수치화

03 Pygame 설정, 텍스트 렌더링

→ 폰트, 이미지 업로드

04 이벤트 루프 및 상태 관리

사용자의 마우스 클릭 입력 감지
→ 클릭된 버튼 위치 확인 / 점수 저장

코드 설계

직관적 UX 개선

맞춤형 결과 이미지 동적 매핑

수치화 했던 결과 나열

→ 이미지 처리

긴 텍스트를 이미지화

→ 가용성 증가, 직관적 개선

MBTI 결과별 이미지 파일 이름 매핑

```
RESULT_IMAGES = {  
    mbti: f"{mbti}.png"  
    for mbti in (  
        "INTJ", "INTP", "ENTJ", "ENTP", "INFJ", "INFP", "ENFJ", "ENFP",  
        "ISTJ", "ISFJ", "ESTJ", "ESFJ", "ISTP", "ISFP", "ESTP", "ESFP",  
    )  
}
```

def load_result_image(mbti):

최종 계산된 MBTI 문자열로 이미지를 동적 로드

filename = RESULT_IMAGES.get(mbti)

if not filename:

return None

이미지 로드 및 화면 비율에 맞춘 스케일링 처리

img = pygame.image.load(str(BASE_DIR /
 filename)).convert_alpha()

... (스케일링 로직 생략) ...

return img

코드 설계

구조적 완성도

FSM(유한 상태 기계) 기반 화면 제어

한 루프 안에서 모든 단계 동시 진행
→ 버그 발생

단계 마다 루프 생성
→ 구조적 한계 극복

3가지 독립적인 화면 상태 정의

PHASE_QUIZ = "quiz"

PHASE_TRANSITION = "transition"

PHASE_RESULT = "result"

메인 이벤트 루프 내부의 상태(Phase) 기반 제어 로직

if phase == PHASE_QUIZ:

draw_background(screen)

퀴즈 화면 UI 렌더링 및 클릭 이벤트 처리

elif phase == PHASE_TRANSITION:

애니메이션 경과 시간에 따른 진행도(progress) 계산

elapsed = pygame.time.get_ticks() - transition_start_ms

progress = min(1.0, elapsed /
TRANSITION_DURATION_MS)

애니메이션이 끝나면 결과 화면 상태로 전이(Transition)

if draw_sneeze_transition(screen, helper_sneeze_img,
progress):

phase = PHASE_RESULT

else: # phase == PHASE_RESULT

draw_background(screen) # 최종 결과 이미지 화면 렌더링

코드 설계

방어적 프로그래밍

데이터 무결성 검증 및 엣지 케이스 방어

응답 회피(전부 3점)
데이터 필터링 알고리즘

무성의 진행 검사 시 처음부터 재시작
→ 결과의 정확도 확보

MBTI 심리검사의 통계적 유의성 및
결과의 품질 확보

```
def dimension_all_threes(pos_ans, neg_ans):  
    """한 지표의 응답이 모두 3(보통)인지 확인하는 검증 함수"""
```

```
    combined = pos_ans + neg_ans  
    if not combined:  
        return False  
    return all(v == 3 for v in combined)
```

```
# 사용자 클릭 이벤트 처리 중 방어 로직 적용  
# 현재 지표의 문항이 모두 끝났을 때 데이터 무결성 검사 진행  
if dimension_all_threes(answers_pos[current_key],  
    answers_neg[current_key], ):  
    reset_quiz() # 모든 응답이 3이면 검사 데이터 초기화, 재시작  
    break
```

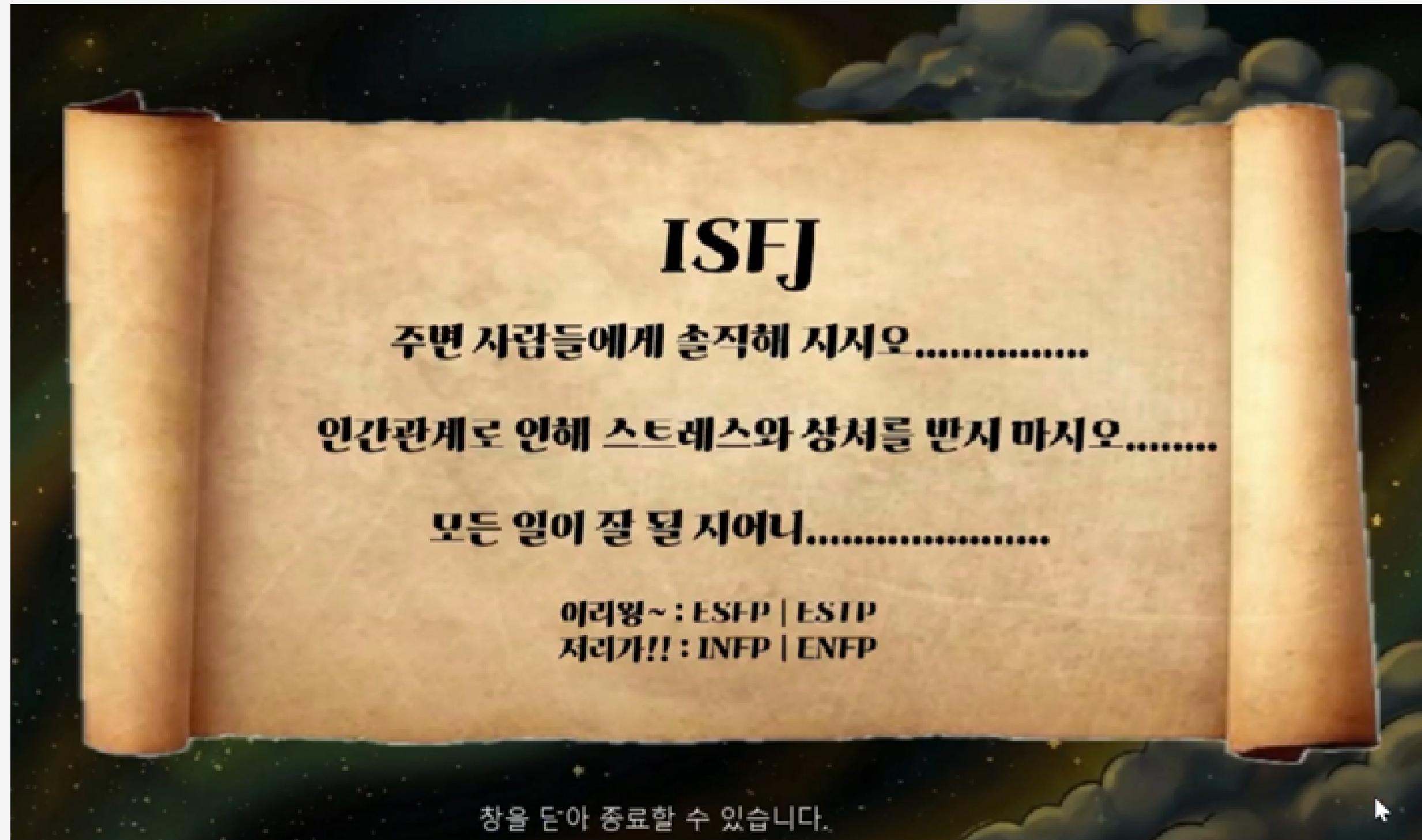
실제 system

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현



실제 system

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현



프로그램 시연

프로젝트 진행 후기

Pygame을 활용한 MBTI 검사 및 유형별 정보 탐색 시스템 구현

01

AI 활용
Programming
도입

02

사용자 중심
UI/UX 구현 경험

03

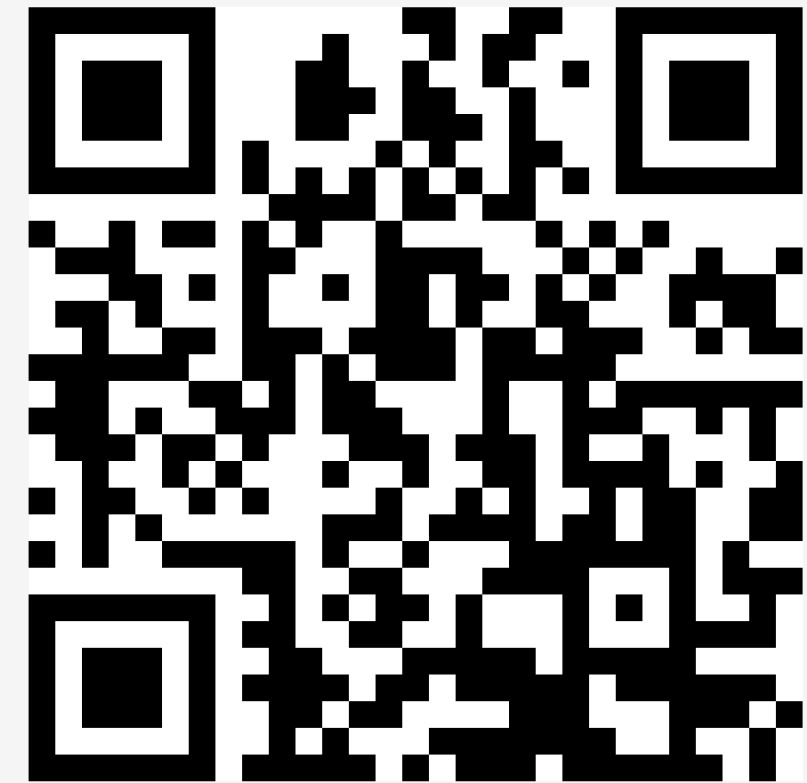
직관적 인터페이스
구성

04

사용자 참여형
로직의 고도화

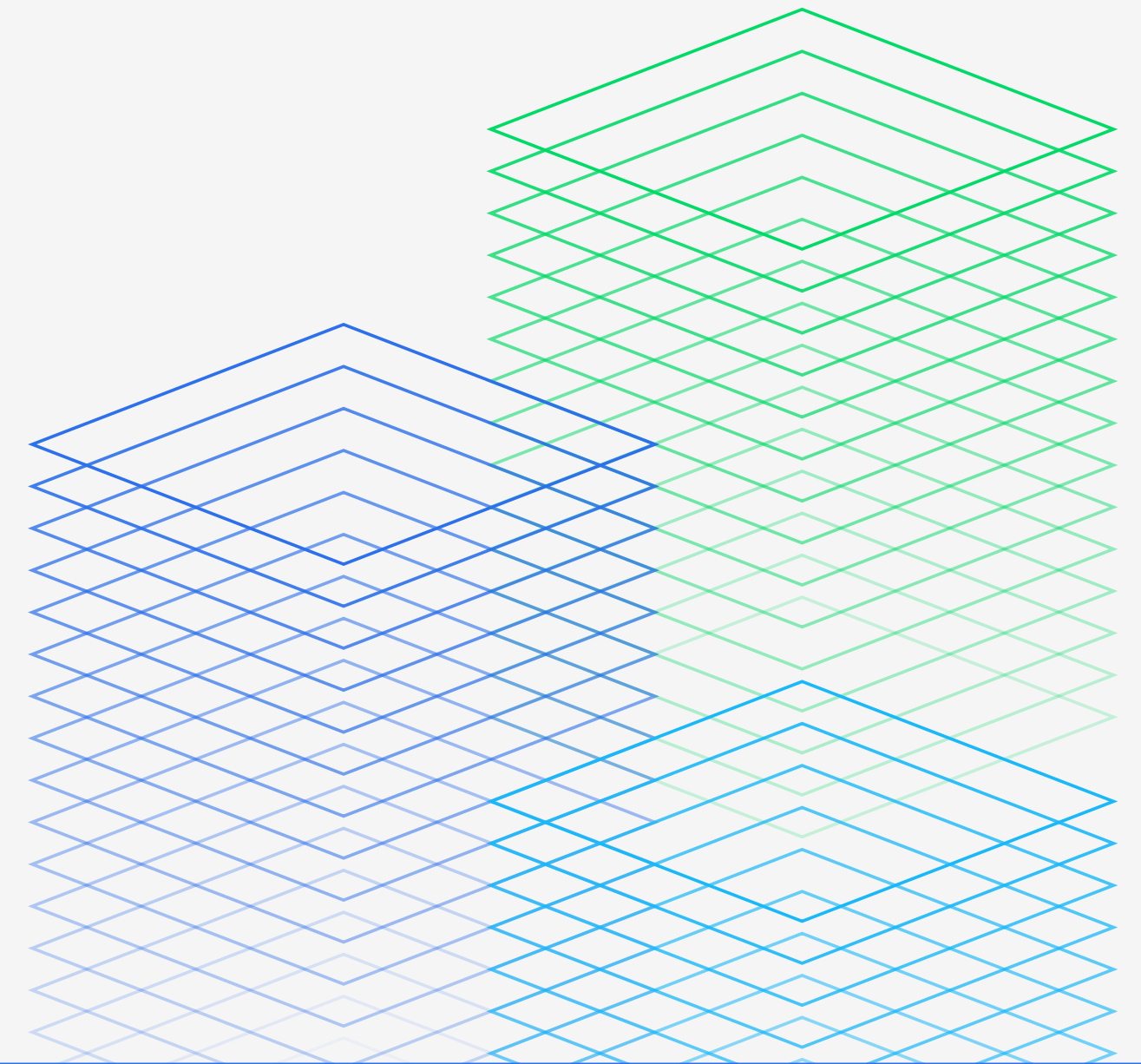
Github URL

https://github.com/aileen49435/03_Team08



20251511 김다연 | 20251536 이금성나 | 20251543 이지현

감사합니다.



Pygame

project를 통해 새로 터득한 정보

(중간 발표 내용 포함 슬라이드)



UI 구현

직관적인 화면 구성 지원
텍스트 및 이미지 기반 system 구현 가능

동작 구조

사용자 입력에 따른 실시간 반응 처리
이벤트 중심 구조 설계 가능

개발 환경

python 기반 환경
낮은 진입장벽으로 활용 가능

콘텐츠 구현

게임 · 테스트형 콘텐츠 제작 용이
2D 인터페이스 제작에 적합